

MongoDB: An Overview

Discover all that you can do with MongoDB, the popular cross-platform, document-oriented database.



MongoDB is a NoSQL database that uses documents to store data in an organised way. Its document data model presents a great way to work with semi-structured data. A distributed systems design intelligently puts data in different servers and locations, scaling it up dynamically. And it can run from different OSes, eliminating vendor lock-in.

In a data driven world we need to save loads of data. MongoDB does this through replica sets and sharding. Figure 1 shows the MongoDB architecture. Let us explore each component in this architecture, and see what happens in the background when the application server hits the MongoDB server to fetch or feed data.

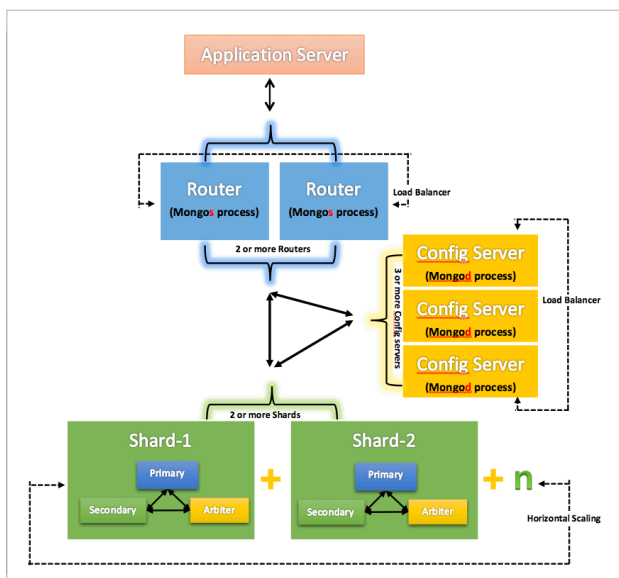


Figure 1: MongoDB architecture (Credits: <https://www.livescript.in/2018/10/sharding-in-mongodb.html>)

Replica sets

Replica sets in MongoDB provide redundancy and high availability. They comprise a group of *mongod* processes that maintain the same data set. Replica sets have been designed for high resiliency — if the primary node goes down, a node from the secondary cluster is automatically elected as primary. These sets can hold 50 nodes in total.

The new primary is selected by voting and meeting some conditions such as:

- Most recent updates from the primary data node
- The latest timestamp and heartbeat status
- History of the connectivity status of other secondary nodes
- User-defined priority

Node types: There are three node types: primary, secondary and arbiter.

- **Primary node:** All the write happens in this node. Only one node acts as the primary node. At times another node may act as primary temporarily, causing the split brain issue. For example, we have six nodes in two data centres, and one node is primary. Due to network issues, connectivity stops between them, causing a network partition. As there is no primary node in one data centre, among the three nodes in it, one is selected as primary. However, having two primary nodes is not recommended. But this is a temporary issue. If the current primary cannot see a majority of voting members, it will step down and become a secondary. Meanwhile only one primary node with { w: "majority" } write concern will be capable of confirming the writes.

- **Secondary nodes:** All the writes done in the primary node are replicated in all the secondary nodes by applying the oplogs. When the primary node goes down, another one is elected as primary from these secondary nodes.

- **Arbiters:** Arbiters are *mongod* instances that are part of a replica set but do not hold data (i.e., do not provide data redundancy). However, they can participate in elections. Arbiters have minimal resource requirements and do not require dedicated hardware.

Consistency levels

- **readConcern:** The readConcern option allows control of the consistency and isolation properties of the data read from replica sets and replica set shards. The levels are: